

Acceso a Datos - Semana 4

UD1. Manejo de ficheros - Binarios y serialización

Objetivo: comprender cómo almacenar información en formato binario y cómo guardar y recuperar objetos Java completos.

1. Texto frente a binario

Los ficheros binarios toman más relevancia que los ficheros de texto en el uso de aplicaciones.

Los ficheros binarios están pensados para ser leídos por un programa, por lo que es necesario conocer los tipos de dato que contienen y su estructura.

Los ficheros binarios almacenan secuencias de dígitos binarios que no son legibles directamente por el usuario, tienen la ventaja de que ocupan menos espacio en disco.

Texto	Binario
Legible por personas	No es directamente legible
Representa caracteres	Representa bytes
Fácil de intercambiar	Puede ser más compacto/estructurado
Ej.: TXT, CSV	Ej.: DAT, imágenes, objetos serializados

Nombre	¿Admite decimales?	Valor mín		Valor máx		Precisión	Ocupa
byte	no	-128	-2^7	127	2^7-1	-	1 byte
short	no	-32768	-2^{15}	32767	$2^{15}-1$	-	2 bytes
int	no	-2.147.483.648	-2^{31}	2.147.483.647	$2^{31}-1$	-	4 bytes
long	no	-9.223.372.036.854.775.808	-2^{63}	9.223.372.036.854.775.807	$2^{63}-1$	-	8 bytes
float	si	-3.4E38		3.4E38		7 cifras	4 bytes
double	si	-1,7E308		1,7E308		16 cifras	8 bytes

2. Flujos de bytes

En Java las dos clases te permiten trabajar con este tipo de ficheros: `FileInputStream` y `FileOutputStream`, trabajan con flujos de bytes y crean un enlace entre el flujo de bytes y el fichero.

```
try (FileOutputStream fos = new FileOutputStream("datos.bin")) {  
    fos.write(65);  
    fos.write(66);  
    fos.write(67);  
}
```

3. Clase FileInputStream:

Los métodos que proporciona la clase FileInputStream para lectura son similares a los visto para la clase FileReader. Estos métodos devuelven el número de bytes leídos o -1 si no lee nada:

int read(): Este método lee un byte y lo devuelve.

int read(byte[] b): Este método lee una cantidad de bytes de datos correspondiente a b.length y los devuelve en b.

int read(byte[] b, int desplazamiento, int n): Este método lee hasta n bytes dejándolos en el array b comenzando en la posición indicada por desplazamiento y devuelve el número de bytes leído.

Ejemplo de lectura en archivos binarios

Un fichero binario o de datos que están formados por secuencias de bytes. Estos archivos pueden contener datos de tipo básico (int, float, char, etc) y objetos. **Para poder leer el contenido de un fichero binario debemos conocer la estructura interna del fichero**, debemos saber cómo se han escrito: si hay enteros, long, etc. Y en qué orden están escritos en el fichero. **Si no se conoce su estructura, podemos leerlo byte a byte con el método read()**, lee un byte y lo devuelve como un int. Si no puede leer, devuelve un -1.

```
String fileName = "archivoBinario.dat";
// Leer datos desde el archivo binario
try {
    FileInputStream fis = new FileInputStream(fileName);
    int num;
    System.out.println("Datos leídos desde el archivo binario:");
    while ((num = fis.read()) != -1) System.out.print(num + " ");
    fis.close();
} catch (IOException e) { System.err.println("Error al leer"); }
```

```
Datos leídos desde el archivo binario:
1 0 3 80 101 112 0 0 0 1 64 31 51 51 51 51 51
```

4. Clase FileOutputStream:

La clase FileOutputStream es muy similar pero para la escritura:

void write(int b): Este método escribe un byte, el byte b.

void write(byte[] b): Este método escribe una cantidad de bytes de datos correspondiente a b.length. Es decir, escribe b.

void write(byte[] b, int desplazamiento, int n): Este método escribe hasta n bytes de la matriz b comenzando por desplazamiento.

5. DataInputStream y DataOutputStream

Permiten escribir y leer tipos primitivos(boolean, int, float, String, etc) respetando un formato binario.

```
try (DataOutputStream dos = new DataOutputStream(new FileOutputStream("persona.dat"))) {
    dos.writeInt(25);
    dos.writeDouble(7.5);
    dos.writeUTF("Ana");
}

try (DataInputStream dis = new DataInputStream(new FileInputStream("persona.dat"))) {
    int edad = dis.readInt();
    double nota = dis.readDouble();
    String nombre = dis.readUTF();
}
```

Importante: el orden de lectura debe coincidir con el orden de escritura.

La clase `DataOutputStream` contiene métodos para escribir tipos primitivos a un stream:

Método/Constructor	Descripción
<code>DataOutputStream(OutputStream out)</code>	Crea un nuevo objeto a partir del stream de salida
<code>void writeInt(int v)</code>	Escribe el entero al stream de salida como 4 bytes
<code>void writeLong(long v)</code>	Escribe el long al stream de salida como 8 bytes
<code>void writeUTF(String s)</code>	Escribe la cadena en el flujo de salida utilizando una representación de UTF-8. El método también escribe la longitud de la cadena en dos bytes antes de la representación de los caracteres.
<code>void writeFloat(float v)</code>	Escribe 4 bytes en el flujo de salida para representar el valor float utilizando IEEE 754
<code>void writeDouble(double v)</code>	Escribe 8 bytes en el flujo de salida para representar el valor double utilizando IEEE 754

El método `writeUTF` es un método que se utiliza para escribir datos de texto en un archivo binario. Primero Escribe dos bytes de información sobre la longitud de la cadena, y luego escribe los caracteres de la cadena. Cada carácter en la cadena se codifica en una secuencia de 1 o más bytes.

Carácter menor que `\u0080`, se codifica como un solo byte

Carácter mayor o igual que `\u0080` y menor que `\u0800`, dos bytes

Carácter mayor o igual que `\u0800`, se codifica en tres bytes

Para la lectura o escritura de un fichero binario:

1. Crear un `File` con el origen/destino de datos
2. Envolverlo en un `FileInputStream/FileOutputStream` para crear un flujo de datos desde/hacia el fichero
3. Envolver el objeto anterior en un `DataInputStream/DataOutputStream` para poder leer/escribir tipos de datos primitivos del flujo de datos.
4. Usar métodos del estilo `writeInt`, `writeDouble`, `readInt`, `readDouble`, etc.

```
File fichero = new File("fichero.dat");
try {
    DataOutputStream out = new DataOutputStream(new FileOutputStream(fichero));
    out.writeUTF("datos");
    out.close();
} catch (FileNotFoundException e) { e.printStackTrace(); }
```

```
boolean aprobado = true;
String nombre = "PRG";
int conv = 1;
double nota = 7.8;
try {
    DataOutputStream out = new DataOutputStream(new FileOutputStream("ejemplo.dat"));
    out.writeBoolean(aprobado);
    out.writeUTF(nombre);
    out.writeInt(conv);
    out.writeDouble(nota);
    out.close();

    DataInputStream in = new DataInputStream(new FileInputStream("ejemplo.dat"));
    System.out.println("Valor leído de aprobado: " + in.readBoolean());
    System.out.println("Valor leído de nombre: " + in.readUTF());
    System.out.println("Valor leído de convocatoria: " + in.readInt());
    System.out.println("Valor leído de nota: " + in.readDouble());
    in.close();

} catch (FileNotFoundException e) { System.out.println("No encontrado"); }
} catch (IOException e) { System.out.println("Problemas al escribir"); }
```

6. Serialización de objetos

Se pueden guardar objetos de tipos primitivos y strings, pero generalmente se pretende guardar objetos más complejos en ficheros binarios. Para ello, estos objetos deben implementar la interfaz serializable, **java.io.Serializable**, la cual no dispone de métodos si no que simplemente indica que esta clase puede ser serializada. La serialización en Java es un proceso inherentemente inseguro, ya que puede ser utilizada para fines malintencionados como la inyección de objetos. Por lo tanto, siempre debes asegurarte de deserializar solo los datos de fuentes en las que confíes y, si es posible, usar alternativas más seguras como JSON, que veremos más adelante.

Para guardar y leer objetos en ficheros binarios usaremos:

Object readObject(): para leer un objeto.

void writeObject(Object obj): para escribir un objeto.

Serializar consiste en transformar el estado de un objeto en una secuencia de bytes para almacenarlo o transmitirlo.

```
public class Alumno implements Serializable {
    private static final long serialVersionUID = 1L;
    private int id;
    private String nombre;
```

```
private int edad;  
}
```

7. Proceso de serialización

1. Crear un File para **almacenar objetos**.
2. Envolverlo en un **FileOutputStream** para crear un flujo de datos
3. Envolver el objeto anterior en un **ObjectOutputStream** para poder **escribir** objetos del flujo de datos.
4. Usar el método **writeObject**, utilizando como parámetro una instancia de un objeto que implementa la interfaz Serializable.

```
Grupo dam = new Grupo("1DAM");  
try {  
    FileOutputStream fos = new FileOutputStream("archivo.dat");  
    ObjectOutputStream out = new ObjectOutputStream(fos);  
    out.writeObject(dam);  
    out.close();  
  
} catch(IOException e) { System.out.println(e); }
```

8. Proceso de deserialización

1. Crear un File para **leer objetos del fichero**.
2. Envolverlo en un **FileInputStream** para crear un flujo de datos
3. Envolver el objeto anterior en un **ObjectInputStream** para poder **leer** objetos del flujo de datos.
4. Usar el método **readObject**, Devolverá un Object que debes convertir al tipo de la clase del objeto, haremos un casting. Dicho método además puede lanzar una excepción adicional del tipo ClassNotFoundException que deberá controlarse.

```
Grupo dam = null;  
try {  
    FileInputStream fis = new FileInputStream("archivo.dat");  
    ObjectInputStream in = new ObjectInputStream(fis);  
    dam = (Grupo)in.readObject();  
    in.close();  
  
} catch(ClassNotFoundException e) { e.printStackTrace(); }  
} catch(IOException e) { e.printStackTrace(); }
```

La clase **ObjectOutputStream** contiene métodos para escribir objetos a un stream:

Método/Constructor	Descripción
ObjectOutputStream(OutputStream out)	Crea un nuevo Objeto a partir del stream de salida
void writeInt(int n)	Escribe el entero n al stream de salida como 4 bytes
void writeLong(long n)	Escribe el long n al stream de salida como 8 bytes
void writeUTF(String s)	Escribe el String s en formato UTF8
void writeDouble(double d)	Escribe el double d al stream de salida como 8 bytes
void writeObject(Object obj)	Escribe el Objeto obj al stream de salida. Ello provoca la escritura de todos los objetos de los cuales obj esté compuesto(y así recursivamente).

Ejemplo de Serialización:

- Vamos a utilizar como ejemplo la escritura y la lectura de objetos en un fichero para gestionar la información de un objeto de la clase **Grupo**.
- Tendremos como clases:
 - **Alumno**: que contiene información individual de un alumno.
 - **Grupo**: que contiene información de todos los alumnos del grupo.

Es necesario indicar que los objetos que se van a guardar serán “serializables”. Por ello, lo declaramos en la cabeceras de las clases.

```
public class Alumno implements Serializable {  
    //cuerpo de la clase  
}
```

```
public class Grupo implements Serializable {  
    //cuerpo de la clase  
}
```

```
Grupo dam = new Grupo("1DAM");  
dam.agregarAlumno(new Alumno("Pep", "1111A", 15));  
dam.agregarAlumno(new Alumno("Tom", "2222A", 17));  
FileOutputStream fos = new FileOutputStream("archivo.dat");  
ObjectOutputStream out = new ObjectOutputStream(fos);  
out.writeObject(dam);
```

Ejemplo de Deserialización:

- Vamos a utilizar como ejemplo la escritura de objetos en un fichero para gestionar la información de un objeto de la clase **Grupo**.
- Tendremos como clases:
 - **Alumno**: que contiene información individual de un alumno.
 - **Grupo**: que contiene información de todos los alumnos del grupo.

Es necesario hacer un casting al utilizar readObject(), ya que dicho método devuelve una instancia de la clase Object.

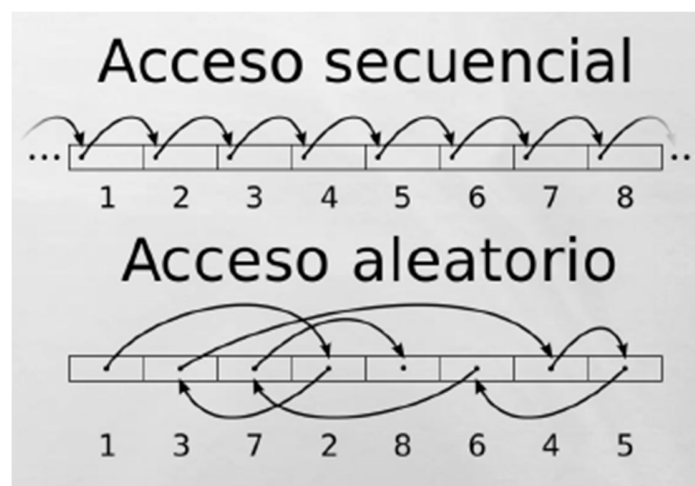
```
Grupo dam = null;
FileInputStream fis = new FileInputStream("archivo.dat");
ObjectInputStream in = new ObjectInputStream(fis);
dam = (Grupo)in.readObject();
for(Alumno a : dam) {
    //muestra el toString() asociado a los alumnos del grupo
    System.out.println(a);
}
```

< 62 > ⋮

9. Clase RandomAccessFile

El acceso secuencial empieza la lectura en el primer byte o el primer carácter o el primer objeto y seguidamente se leían los siguientes 1 detrás de otro hasta llegar al final del fichero. Si se quiere acceder a un dato que está en la mitad del fichero es necesario leer antes todos los anteriores.

Igualmente, cuando se escriben los datos en el fichero se van escribiendo a continuación de la última información almacenada. No es posible realizar inserciones entre los datos.



Java dispone de la clase **RandomAccessFile** que dispone de métodos para acceder al contenido de un fichero binario de forma aleatoria y permite posicionarse en un punto concreto del mismo. Los datos se almacenan en registros de tamaño conocido, por lo que nos podemos mover de uno a otro para leerlos o modificarlos.

Es importante destacar que esta clase no es parte de la jerarquía `InputStream/OutputStream`, ya que su comportamiento es totalmente distinto puesto que se puede avanzar y retroceder dentro de un fichero.

Se dispone de 2 constructores para crear el fichero de acceso aleatorio y ambos pueden lanzar la excepción `FileNotFoundException`:

`RandomAccessFile (String nombreFichero, String modoAcceso)`

`RandomAccessFile (File nombreFichero, String modoAcceso)`

el argumento `modoAcceso` puede tomar los siguientes dos valores:

`"r"`: abre el fichero en modo solo lectura. El fichero debe existir.

`"rw"`: abre el fichero en el modo lectura y escritura. Si no existe el fichero, se crea.

Una vez abierto el fichero se puede usar como `DataInputStream` y `DataOutputStream` utilizando los métodos `read()` y `write()`.

La clase `RandomAccessFile` Maneja un puntero que indica la posición actual en el fichero. Cuando en el fichero se crea el puntero se coloca en cero, apuntando al principio del mismo.

Las sucesivas llamadas a los métodos `read` y `write()` ajustan el puntero según la cantidad de bytes leídos o escritos.

Los métodos más importantes son los siguientes:

`long getFilePointer()`: Devuelve la posición actual del puntero del fichero.

`void seek(long posición)` : coloca el puntero del fichero en una posición determinada desde el comienzo del mismo.

`Long lenght()`: devuelve el tamaño del fichero en bytes. La posición `lenght()` marca el final del fichero.

`Int skipBytes(int desplazamiento)`: desplaza el puntero desde la posición actual el número de bytes indicados en desplazamiento.

La función `seek` se utiliza para mover el "puntero de archivo" a la ubicación deseada. Por ejemplo, para cada entero, JAVA utiliza 4 bytes, por lo que para saltar al sexto entero del fichero, debemos desplazarnos 20 bytes (4×5) y leer los siguientes 4 bytes para obtener el sexto entero:

```
try {
    RandomAccessFile raf = new RandomAccessFile("random.dat", "rw");

    for (int i = 0; i < 10; i++) {
        raf.writeInt(i);
    }

    raf.seek(5 * 4);
    int sexto = raf.readInt();
    System.out.println("El sexto entero es: " + sexto);

    raf.close();
} catch (IOException e) { System.out.println("ERROR"); }
```

RandomAccessFile Puede utilizarse con ficheros de texto. Sin embargo, hay que tener cuidado ya que no tiene información de las codificaciones de caracteres ni de los saltos de línea. Está diseñado para trabajar a nivel de bytes. Esto puede causar problemas si, por ejemplo, estás utilizando una codificación de caracteres que utiliza múltiples bytes por carácter, como UTF-8.

Sin embargo, puede ser útil para trabajar en ciertas situaciones. Por ejemplo, si estás trabajando con un fichero de texto que utiliza una codificación de un solo byte por carácter (como ASCII). Y no necesitas preocuparte por los saltos de línea.

Si trabajamos con un fichero de texto que utiliza una codificación de múltiples bytes por carácter, o se necesitan manejar correctamente los saltos de línea, debemos usar otras clases, BufferedReader, BufferedWriter, etc.

10. Al terminar la semana debes saber

- Distinguir texto y binario.
- Escribir y leer tipos primitivos.
- Comprender el orden de lectura/escritura.
- Serializar y deserializar un objeto.
- Serializar una colección de objetos.
- Entender Serializable, serialVersionUID y transient.